O'REILLY®

Cloud Native Data Security with OAuth

A Scalable Zero Trust Architecture

Gary Archer,
Judith Kahrer &
Michał Trojanowski

Praise for *Cloud Native Data Security with OAuth*

This book is a fantastic guide and resource to anyone looking to implement OAuth within their Cloud Native architecture. The authors have done an excellent job covering the basics but also tackling the more complex and advanced areas. They make the topic accessible and relevant to readers wanting to understand the topic in depth, and I'd highly recommend it.

—Matthew Auburn, coauthor of *Mastering API Architecture*

This book offers a comprehensive and practical guide to securing APIs in cloud-native environments. It strikes an excellent balance between theory and practice, providing clear explanations and code examples. I highly recommend this book to anyone involved in building, deploying, or securing APIs in a cloud-native context, as it equips you with the knowledge and tools needed to safeguard your applications and data in today's dynamic and distributed systems.

—Narang Dixita Sohanlal, software engineer, Google

This book is a must read for anyone looking to adopt OAuth standards. The authors are subject matter experts on the topic and have provided an excellent overview combined with practical advice and examples.

—Veena Rajarathna, product manager focused on API security, Kong Inc.

Cloud Native Data Security with OAuth

A Scalable Zero Trust Architecture

Gary Archer, Judith Kahrer,
and Michał Trojanowski

Cloud Native Data Security with OAuth

by Gary Archer, Judith Kahrer, and Michał Trojanowski

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Simina Calin
Development Editor: Virginia Wilson
Production Editor: Clare Laylock
Copyeditor: Krsta Technology Solutions
Proofreader: Vanessa Moore
Indexer: Judith McConville
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: Kate Dullea

March 2025: First Edition

Revision History for the First Edition

- 2025-03-06: First Release

See http://oreilly.com/catalog/errata.csp?isbn=9781098164881 for release details.

This work is part of a collaboration between O'Reilly and Curity. See our statement of editorial independence.

978-1-098-16488-1

[LSI]

Foreword

Our team successfully coded an advanced OAuth and OpenID Connect server that supported the complex security standards covered in this book. In doing so, we gained deep insights into these standards and productive ways to use them. Despite our years of hard work, we quickly realized that it would be for naught if we were unable to help others understand where our product fit in the confluence of digital identity and API security.

As organizations digitalize their services, they inevitably need to expose data as web services. These APIs are consumed by mobile apps, single-page applications, and traditional websites. In effect, this makes those APIs platforms for new digital processes, business models, and experiences. This metamorphosis to digital natives requires organizations to authenticate users and authorize access to data APIs. These are difficult problems that the OAuth and OpenID Connect specifications were designed to help solve. It is paramount that those seeking to adopt digital business also embrace and deploy these standards.

To explain how to do so, we needed to add to our team people that had not only mastered these technologies but also had an ability to teach them to others. Those people had to be skilled communicators, engaging instructors, and fluent writers. We found these qualities in Gary, Judith, and Michał and were immediately fond of their desires to uplift the industry through knowledge sharing. All three of them have a passion to teach complex topics and articulate the value behind them in ways that normal software teams can understand.

As we worked together, the new team frequently used cloud native technologies to demonstrate OAuth standards and back up the theory. We realized that there was another convergence of technologies that needed to be taught. The team's learning resources explained how digital identity and APIs were coupled while also showing how these technologies could be used in cloud computing as native components. By protecting data exposed in meshes of microservices, repeatable patterns emerged from their work that spanned use cases and industries. As we learned new techniques from them, we understood that this critical knowledge also needed to be disseminated.

When the authors proposed writing a book on these topics as a part of their day jobs, we agreed without hesitation because Curity's mission is to play a part in making the internet safer by, among other things, spreading knowledge on identity topics. Because of our roles at Curity, we had the unique opportunity to read each chapter as the authors wrote them. With each, we appreciated their advocation of design patterns, adherence to best practices, and clear communication of protocols devoid of product-related promotions.

With the knowledge they captured in the chapters you're about to read, the authors not only explain how digital identity and API security intersect but also how to deploy solutions to such problems in cloud-native environments. If you haven't performed such deployments before, this book should help you to understand the approach and how to deal with multiple components and endpoints, to help you ensure that your installations are secure, modern, and easy to operate.

The book uses both OAuth and cloud native in a technology-neutral and vendor-neutral manner. The tools and techniques remain relevant even if you do not operate in a cloud native environment. Ultimately, it is about serving APIs in the best ways, which in turn serves your business and users. In many organizations, you need to consider regulatory restrictions, multicloud strategies and migrations from legacy identity infrastructure to enable OAuth. Cloud native can help you with these goals. As the authors will show you, integrating OAuth and cloud native is not as hard as you may think and comes with many benefits.

With this said, we recommend this book to you, and leave you now in the hands of the authors.

Travis Spencer and Jacob Ideskog,
Cofounders of Curity

Preface

When you build digital solutions, your users run applications that access data over the internet. These applications can run on browsers, mobile devices, servers, or any other platform. They have one thing in common, though: they are API clients, and that means they call APIs to access data. APIs must enable secure data access because the data they serve may include intellectual property, personal data belonging to citizens or users, or information belonging to business partners. To protect data and access to it, you must secure both APIs and the application fetching the data. Such security requirements can originate from compliance and regulatory initiatives. One example is the European Union's General Data Protection Regulation (GDPR), which governs user privacy and consent.

Managing data security and privacy in APIs and API clients is a difficult problem that requires an architectural solution. Your APIs must process user identities correctly so that each user gains access to only the correct API resources. To achieve that, you must apply business authorization in APIs. The exact authorization rules depend, of course, on your business use cases. Ideally, you ensure that every API request from a client includes an unforgeable and least-privilege API message credential. This credential conveys business permissions. You first verify such a credential in the API before you enforce your particular rules. When you protect every API request in this manner, for both external and internal requests, you get a zero trust API architecture.

When implementing API security, we assume you want certain architectural qualities, including reliability, scalability, and a productive developer experience. Your architecture should enable you to meet all of these nonfunctional requirements in addition to security. We show you how to achieve such an architecture using solid design choices. We demonstrate a separation of concerns approach to externalize difficult areas from your application code. For example, you can host security components on a cloud native platform with modern clustering and elasticity features to meet your high-availability requirements. Similarly, you can manage cryptography and other security plumbing using specialist endpoints and libraries. Your choices should enable a future-proof setup and scale to many applications, APIs, and teams.

Why We Wrote This Book

We (the authors) all work at Curity, a company specialized in identity and access management for APIs. In our day-to-day jobs, we teach OAuth and related technologies to a wide audience. We base some of our online material on the Curity Identity Server, yet the majority of our content is not

product-specific and explains a standards-based architecture. Our work involves presentations, videos, online articles, and code examples as well as investigations and explorations in the space of secure authentication and API access. Although cloud native is the state of the art when it comes to deploying APIs, we noticed that resources on cloud native security mainly focus on the infrastructure. We acknowledge that infrastructure security, such as encryption, is important to secure data, and so are processes, policies, and governance. Authorization is the piece of the puzzle that we happen to be experts in.

We wrote this book because we believe that OAuth has major architectural benefits for applications and APIs, yet it is widely misunderstood. This prevents organizations from making the right choices and fully realizing the benefits. In particular, we believe that access tokens are often used suboptimally. When you design access tokens correctly, they enable you to set security boundaries and control authorization to your secured business resources in a way that benefits all kinds of organizations—from small to very large. By combining a centralized token issuer with smart authorization techniques, you can monitor, audit, and govern resource access at any scale.

We explain the main security requirements for APIs and API clients with regard to authorization and show how OAuth helps you to meet them. Part II is the core of the book and we recommend that you take special care when reading it. We think that if we can help you to understand how identity and business components work together, you will be able to make the right choices and fully benefit from OAuth for a secure and scalable solution. We explain a sequential journey from the viewpoint of developers and organizations building digital solutions. We provide theoretical content along with practical content to back up the theory.

Who This Book Is For

The content is aimed at any developer, architect, or site reliability engineer interested in API security. You may work with APIs that already have some basic security but want to learn how to update to a zero trust architecture. Preferably, you have an intermediate-level knowledge of APIs and API security.

We use Docker and Kubernetes to provide examples that implement our security designs. Thus, if you are familiar with containers, you will get a more hands-on experience from the book. However, you can apply the concepts we describe to any cloud native platform.

The book is detailed in places, so you should be prepared to absorb a considerable amount of content on identity topics. Above all, you should be willing to adopt a separation-of-concerns approach to software engineering.

What You Will Learn

You will learn how to secure APIs and clients using the OAuth 2.0 authorization framework. We will present designs, patterns, and best practices that help you to enforce business permissions in APIs. You will learn how to perform the following tasks:

- Design least-privilege access tokens.
- Operate an authorization server.
- Issue access tokens to API clients, then send them to APIs.
- Validate access tokens in APIs, then authorize access to business data.
- Implement a code flow in clients, to enable many ways to authenticate.
- Manage token confidentiality for internet clients.
- Follow best practices for browser-based applications, to limit damage if there is a cross-site scripting (XSS) exploit.
- Implement OAuth 2.0 for platform-specific applications, to secure both desktop and mobile applications.
- Manage client-specific security differences in an API gateway.
- Combine OAuth 2.0 with infrastructure security.
- Keep application code portable and standards-based.
- Deal with error conditions and ensure reliable end-to-end flows.

Cloud Native Environments

We decided to set this book in a cloud native environment because we believe that such an environment provides the optimal features for managing secured APIs.

> Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.
>
> —CNCF Cloud Native Definition v1.0

In the context of this book, the relevant technologies for the security architecture, as well as the code examples, are containers, services meshes, and microservices. We do not make any assumptions about whether you run a public, private, or hybrid cloud. You will be able to apply the concepts that we present in this book no matter the type of cloud.

We like the choice, portability, and future-proofing that cloud native platforms provide. We think you should utilize these features not only for your APIs but also for supporting (security) components. Security components are critical, so they must be highly available. Therefore we explain some operational behaviors in addition to security.

What is really great about cloud native is the fact that it provides rich options for us to demonstrate end-to-end security on a standalone computer. Ultimately, though, you can apply the design patterns that we describe to other types of environments. When possible, we base solutions on published standards and best practices. Therefore you should be able to benefit from the content we provide, regardless of the hosting platform you use or your technology preferences.

What This Book Is Not

This book is not a complete guide to cloud native security. We won't teach you cloud native and Kubernetes from the ground up, but we will show you how to configure cloud native security components on a development computer, within a Kubernetes cluster. For the best understanding, you should have some existing knowledge about running applications in Kubernetes.

We also do not provide a comprehensive reference that covers all aspects of OAuth 2.0. We will introduce you to OAuth 2.0 and you do not need prior experience with the protocol. We want to focus, however, on practical aspects of designing a security solution based on OAuth 2.0, instead of explaining every flow step by step or describing every specification from the OAuth 2.0 family.

We will skip in-depth examinations of low-level security. For example, we will inform you about the current recommended algorithms for signing tokens, but we will not explain these algorithms or their robust cryptography. We also do not cover how to secure operating systems, containers, databases, or other infrastructure.

Using Code Examples

We supplement the book's theory with code examples that you can download from the book's GitHub repository. Using code examples is entirely optional but may be useful if you want a working implementation to compare against. The examples meet some difficult security requirements using standards-based code that externalizes difficult security from applications. Some code examples are fairly complex, since they use a highly distributed architecture, and you may need to take time to study the code and *README* documents.

Each code example provides an end-to-end solution that you can run without restrictions on a Windows, macOS, or Linux development workstation. When applicable, you can also deploy examples to a local Kubernetes cluster. Once you understand the local deployment, you can use the same techniques to deploy the examples to a remote cloud native environment, such as the Kubernetes platform provided by your cloud provider.

We want you to be able to get up and running quickly, so we use the free version of the Curity Identity Server as the default authorization server. We also use various other cloud native security components and respected security libraries. When needed, you can replace security components with alternative ones, as long as they support the required standards and extensibility. Because of variations in implementations of standards, we cannot guarantee that code examples will immediately work with every replacement product.

Example APIs and clients follow current best practices at the time of writing. Most code examples and code snippets use TypeScript, as a simple yet expressive language. On mobile platforms we use languages specific to the platform, namely Kotlin and Swift. You can implement the same patterns in alternative technology stacks.

If you have a technical question or a problem using the code examples, please email *support@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Cloud Native Data Security with OAuth* by Gary Archer, Judith Kahrer, and Michał Trojanowski (O'Reilly). Copyright 2025 Curity AB, 978-1-098-16488-1."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

Terminology

We use the term *OAuth* to represent the modern OAuth 2.0 authorization framework, first defined in RFC 6749. OAuth 2.0 has strong technology support in all current API, web, and mobile technology stacks. OAuth 2.0 supersedes the OAuth 1.0 protocol from RFC 5849, which was a more limited solution with fewer architecture benefits. Consequently, we do not cover OAuth 1.0 but always mean OAuth 2.0 when we refer to *OAuth*.

API is an overloaded term in the IT world. In the book, we will mainly use the term *API* to represent Web APIs that expose data to various types of clients, and we use the terms *APIs* and *microservices* interchangeably. We use the term *endpoint* to represent an entry point to a concrete function in your APIs or a security component. We use the terms *clients* and *applications* to represent API clients. APIs

can also act as clients to other APIs. Our primary focus is to apply the OAuth 2.0 framework in a way that best serves APIs and clients.

If you work with Kubernetes, you might see the term *API* used in conjunction with the Kubernetes configuration or control plane. When we use Kubernetes terms that conflict with the normal world of APIs and clients, we clearly explain the context.

This Book's Structure

Many OAuth books start with user logins, which are often the most visible part of OAuth. However, it is common for newcomers to OAuth to get logins working and then run into deeper problems with APIs later. To prevent this type of outcome, we follow an API-first approach. Part I begins with an introduction that explains the business rationale behind OAuth 2.0 and cloud native in Chapter 1, "Why Do You Need OAuth?". Next, Chapter 2, "OAuth 2.0 Distilled", provides an overview of how OAuth works. Chapter 3, "Security Architecture", then explains the building blocks for integrating with OAuth. Finally, Chapter 4, "OAuth Data Design", shows how to approach data design so that your data supports integrating with OAuth.

Part II dives deep into access tokens and APIs. This shows you how to design access tokens, how to write code to secure access to data in APIs, and how to avoid common security pitfalls when working with tokens. You will also learn how to use API gateway design patterns in a Kubernetes setup.

Part III explores OAuth and its components in a cloud native environment, how you can combine features of OAuth and the cloud native platform, and how you operate cloud native OAuth components. It also shows how to get workload identities and other advanced setups running on a developer workstation.

Finally, Part IV covers clients and user authentication. It includes web and mobile code examples that follow current OAuth security best practices. Chapter 14, "User Authentication" covers user authentication. The chapter explains a few topics that comprise modern authentication, including secure user registration and cryptography-backed user logins that also provide a friendly login user experience. We also cover migration of users from an older to a newer authentication method while keeping the user identity the same for APIs.

Some chapters are heavy on theory and others are code-centric. You should feel free to switch between chapters when you want to run some code. For example, after reading Chapter 2, you may want to get a code flow working. To do so, you could jump ahead to Chapter 12, "OAuth for Native Applications", on platform-specific applications and run the simplest type of OAuth client, a console application. For the most complete understanding, though, we recommend then continuing by reading all chapters in sequence.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **TIP**
>
> This element signifies a tip or suggestion.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

O'Reilly Online Learning

> **NOTE**
>
> For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

https://oreilly.com/about/contact.html

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/cloud-native-data-oauth*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly-media*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

Acknowledgments

This book was a cooperative project. Not only did it include three authors but many people in the background that got us to where we are. Some people helped before we even got properly started. One of them was James "Jim" Gough who helped us establish the contacts with O'Reilly at the very beginning. The world may have missed out on a good book if it wasn't for you, Jim! Another person who was involved early was Sunny Wear who, once submitted, reviewed our proposal for O'Reilly. Thank you, Sunny, for believing in our idea and recommending it to O'Reilly so that we could publish the book.

We also want to thank our employer Curity for backing the book from the very first idea to the final chapters and beyond. Special thanks to Jacob Ideskog and Travis Spencer for their foreword and continuous feedback during the process. It meant a lot for us to get that support from our employer!

We also want to thank all the technical reviewers, in alphabetical order, Matthew Auburn, Anders Eknert, Narang Dixita Sohanlal, Aaron Parecki, Veena Rajarathna, and Mukund Sarama for their valuable input to further shape the content. Thank you all for critically reflecting on our statements with regard to your expertise, correcting our mistakes, amending missing pieces, and improving

wording. Your feedback, questions, and suggestions helped us to reflect upon and improve our message. The book wouldn't be what it is without you! Many thanks also to our editor, Virginia Wilson, for guiding us through the process, making sure we stayed on track and got the feedback we needed.

Last but not least, thank you to all the people who help to promote the book and spread the word so that readers find it. We feel very grateful to be surrounded by such a great community. Thank you all so very much!

Part I. Introducing Cloud Native OAuth

Chapter 1. Why Do You Need OAuth?

Around 2010, if you were part of a team developing digital solutions, you would probably have built a website that exposed data to a single client, the browser. In those days, users authenticated with a username and password, after which the website issued an authentication cookie. This cookie secured calls to the backend and included the user identity and an expiry timestamp. The backend then enforced security rules using the cookie user identity, such as preventing user A from accessing user B's data.

To meet today's business needs, you use a more modular system. A backend platform typically consists of multiple APIs, which serve data to web, mobile, and business-to-business (B2B) clients. As with websites, you need a way to authenticate and authorize requests to and between APIs. While cookies still qualify for authenticating requests in certain scenarios, you need a solution that works for a variety of clients. In addition, many organizations operate a large number of APIs and clients, so you also need a solution that scales. This is where OAuth comes into play.

OAuth provides the protocols and tools to implement consistent, scalable access controls in APIs. At its core lies the access token that conveys permissions to access APIs and consequently (business) data. APIs are now business products, whose exposed data generates the business value. Hardened API access is not a technical but a key business concern. OAuth allows you to customize API authorization to meet business requirements.

In this chapter, you will first learn the main steps we recommend for enabling modern API-first security, with a primary focus on correct authorization. We explain how OAuth 2.0 enables this authorization and we recommend applying it to every API request. We call this zero trust API security. To enable the right security, you need components that support your APIs and applications. You also need to operate both APIs and supporting components. We explain how cloud native platforms provide

some leading operational capabilities, including advanced infrastructure security. You will see how your technical choices play an important role in determining whether you achieve the best business outcomes.

API-First Security

Security for a digital business is a multifaceted topic with many best practices, including the following:

- Follow a secure development lifecycle to protect against threats like SQL injection.
- Secure the software supply chain using techniques such as vulnerability scanning.
- Run backend components on secure operating systems using a low-privilege service account.
- Expose data to the internet using Transport Layer Security (TLS).
- Apply network protection using proxies or firewalls.

Yet these strategies alone are insufficient to secure business data. For that, you need a security architecture that focuses on your applications, APIs, and users. A key behavior of such an architecture is the principle of least privilege. You should grant applications only the API permissions that they strictly need, and each API must enforce those permissions. At the same time, you should use hardened security for user authentication but also provide a modern and friendly user experience. What is more, the security behaviors should scale to many APIs and clients without a linear increase in complexity. To achieve that, use the OAuth 2.0 authorization framework.

What Is OAuth 2.0?

We explore the details of the OAuth 2.0 protocol in Chapter 2, but first we want to provide a high-level overview of its primary behaviors. As the name suggests, OAuth is centered on authorization. At its core, it uses an API message credential, the access token, based on which APIs can enforce authorization. The framework was designed to meet the security needs of any type of API client, including browser-based applications and mobile applications, and to work in any technology stack. User privacy and consent are a core part of the design, and users can also authenticate in many possible ways.

There are a number of benefits when leveraging OAuth. First, you avoid homegrown security solutions because you can build upon best practices that many experts have vetted. You can write simpler code and reduce the risk of security flaws. Second, an OAuth architecture is highly extensible and scalable. For example, you can reconfigure user logins in many possible ways, without needing to change any code in clients or APIs. Finally, since OAuth is a well-established standard, there are many security libraries that implement the protocol, which helps to cut the time to market for your secure solution.

OAuth 2.0 was introduced in the RFC 6749 specification, published in 2012. It introduces a component called the authorization server. This component externalizes most low-level security from clients and APIs. Before calling APIs, the client must get an API message credential, the access token. As part of that process, the authorization server authenticates the user (if applicable) before it returns an access token to the client. The client then sends the access token to APIs, typically through an API gateway. APIs only accept requests if they include a valid access token. Figure 1-1 shows an end-to-end flow where the user authenticates before the client receives an access token that it forwards to APIs.

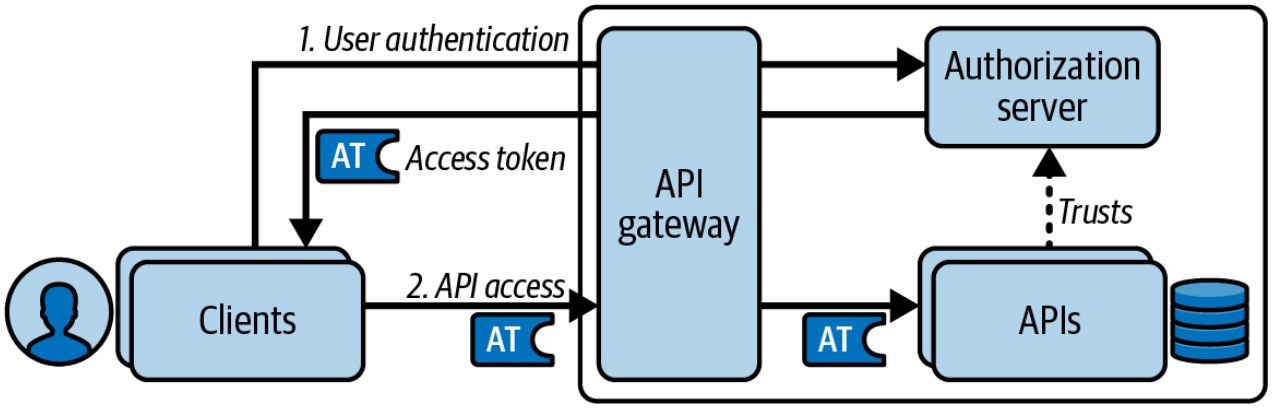

Figure 1-1. OAuth interaction between clients, the authorization server, and APIs

The user login is often the most visible part of OAuth, yet the heart of OAuth is the access token. The access token can provide the user identity and other secure values to APIs in a cryptographically verifiable and auditable manner. You design access tokens in terms of business privileges to APIs. You should customize the access token for each client to only give them least-privilege API access. Since every business is different, there is no fixed way to design access tokens. Similarly, you can choose from many forms of user authentication provided by your authorization server. We explain your choices for access token design in Chapter 6 and your choices for authentication in Chapter 14.

Business authorization is part of your domain logic. Therefore, you usually implement it in the API code alongside other domain logic. On every request to APIs and microservices, the API must validate the access token and authorize access using the received token data. With the access token, you can forward the user identity and other secure values between microservices while keeping the user identity verifiable and auditable so that upstream microservices can also implement security correctly.

There are various other application security frameworks, though most of them provide only user authentication. If you use an authentication framework, you eventually need to invent an API message credential to implement authorization. This results in homegrown solutions that are hard to maintain because of their complexity and reduced interoperability. OAuth 2.0 focuses on API authorization. With OAuth, you can use interoperable security standards for end-to-end flows and base your solution on the work of security experts. This work includes API, client, and user behaviors. Finally, OAuth lets you implement security that scales to many clients and APIs.

OAuth is a framework rather than an out-of-the-box solution. How you apply the framework varies depending on your business requirements for authorization, user authentication, and other security

areas. More generally, OAuth is a family of specifications, each mapping to use cases for software organizations. Implemented correctly, OAuth not only improves your security architecture but also simplifies it by externalizing security complexity from APIs and clients.

For all of these reasons, adopt OAuth as your security design if you have an API architecture with user-facing clients. We see it as a security strategy that can work for any organization. We also recommend a particular implementation approach to API security called the zero trust architecture, where we define zero trust in terms of business permissions to access your APIs.

Zero Trust Security

Traditionally, security controls focused on the perimeter of the infrastructure. In that model, a strong border divides the infrastructure into external and internal parts. It assumes that internal parts are trustworthy and thus focuses only on external security threats. Cloud computing blurs the border, as organizations commonly use cloud services for internal functions. Consequently, the perimeter approach is insufficient for securing a digital business nowadays. Modern API security must therefore address both internal and external threats.

A zero trust approach does not assume any implicit trust, for example, based on infrastructure rules such as internal network addresses. Zero trust means that you should use explicit trust and not assume that requests come from a certain client or user. With regard to API security, this implies that APIs must always verify the caller. To understand the problems with implicit trust, let's start by examining the threats inherent in perimeter security.

APIs with Perimeter Security

When organizations first built web APIs, it was common to adopt a perimeter security approach, where the perimeter represents entry points into the backend cluster called by internet clients. Figure 1-2 shows an example where the website implements strong security by requiring browser clients to send a secure encrypted cookie containing a user ID.

During the processing of web requests, the website calls internal APIs that are not exposed to the internet. The website uses weak security for internal connections. In the example, the website uses an API key for the API message credential and forwards the user identity to APIs in a plain HTTP header.

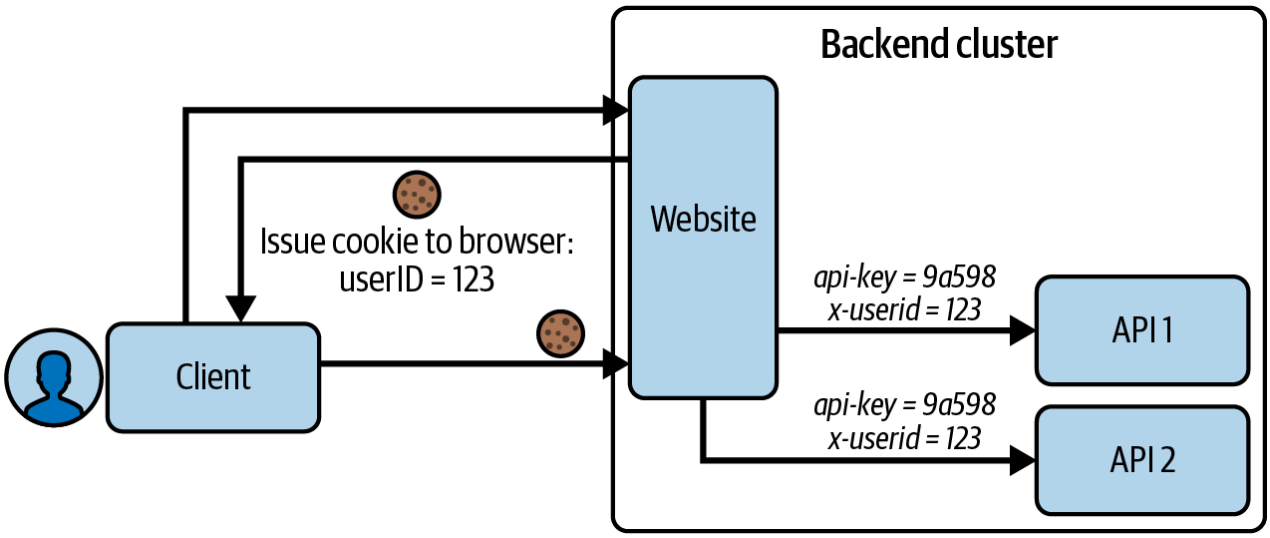

Figure 1-2. Internal APIs with weak security

There are several problems with this design. First, a malicious party inside the network might be able to steal and reuse the API key. Second, you might only rarely renew API keys, in which case an attack could continue against the APIs for a long time. Finally, the user ID is a sensitive value that you need to protect. A malicious party should never be able to bypass security by injecting their own user ID, to get data for another user. With this security design, target APIs cannot know whether sensitive values that they received are correct.

A first attempt to improve the internal API security might rely on infrastructure. Let's examine this type of solution next so that you understand the problems infrastructure cannot solve.

APIs with Infrastructure Security

You can improve API security by securing internal connections with the help of infrastructure security, such as the mutual TLS (mTLS) that cloud native service mesh solutions provide. Figure 1-3 shows an updated example where the entry point website continues to implement strong security by requiring a secure cookie. The website then interacts internally with microservices using a client certificate credential.

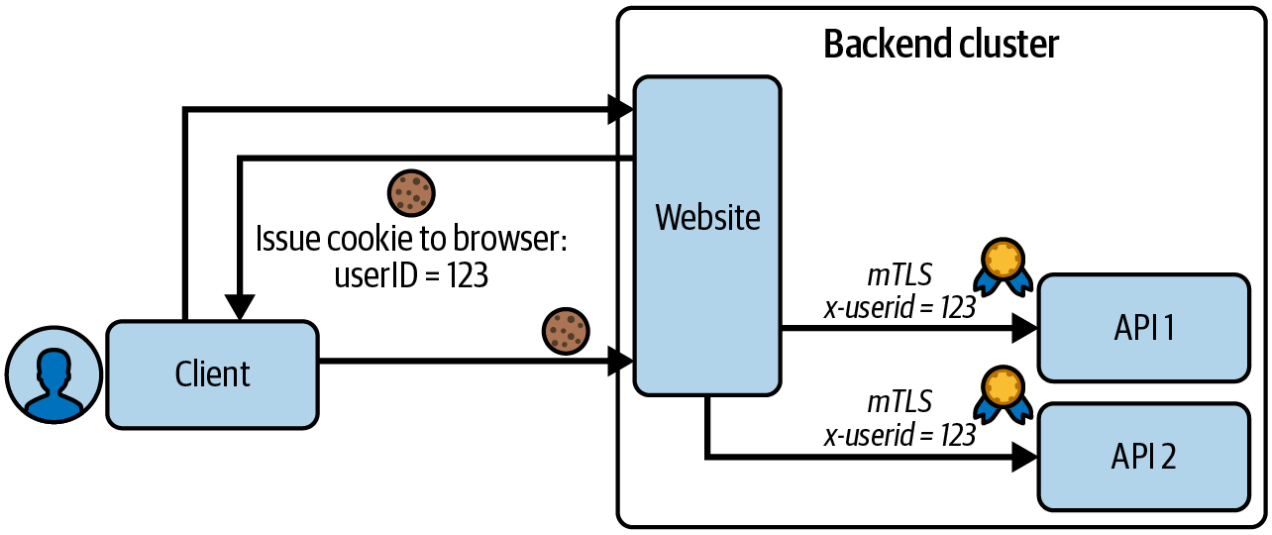

Figure 1-3. Internal APIs with infrastructure security

Adding infrastructure security to protect calls between microservices is certainly an improvement over perimeter security since it is now more difficult for a malicious party to call internal APIs. The stronger connection does not deal correctly with authorization, though. Instead it is an authentication-only solution between backend components. Sensitive values, such as user IDs, continue to be sent in HTTP headers. The API cannot verify that it receives a correct user identity. A malicious or misbehaving API client in the environment could call the API with incorrect HTTP headers and access unauthorized data. To avoid that, we recommend a zero trust OAuth implementation using token-based security. Let's look at that next and explain its benefits.

APIs with Token-Based Security

You can easily enforce explicit trust with OAuth. For that, require that every call to every API endpoint contains an access token with cryptographically verifiable information about the caller. Every API needs to validate the access token to verify its integrity and to ensure that the access token meets that particular API's security requirements. If a token is altered or is not valid for other reasons, the API must reject the request. This means that the API accepts requests only if they contain a valid access token. The trust is explicit because all internal APIs validate the token on every request.

Access tokens are your API credentials. Design them in a way that enables APIs to enforce least-privilege access. First, you can design access tokens to restrict access by business area. Next, you can include values in the access token that your APIs use for authorization. These can be user attributes or runtime values such as the authentication strength. This provides the most powerful ways for your APIs to authorize requests. Finally, assign access tokens a short lifetime to limit the impact of any potential misuse.

When an API validates access tokens, it must reject any requests containing altered or expired tokens. Otherwise, the API trusts the identity attributes in the access token and uses them for business authorization. When implemented correctly this approach provides zero trust in terms of your business. The access token payload in Example 1-1 shows some example attributes.

Example 1-1. Access token payload

```
{
    "sub": "556c8ae33f8c0ffdd94a57b7eab37e9833445143cf6847357c65fcb8570413c4",
    "customer_id": "16771",
    "iss": "https://login.example.com",
    "aud": ["api.example.com"],
    "scope": "products",
    "membership_level": "1",
    "level_of_assurance": 3,
    "exp": 1721980485
}
```

In Chapter 5 we explain the meaning of the standard fields of an access token, such as the `iss`, `aud`, `sub`, and `scope` fields shown in Example 1-1. We also show how you can write API code to correctly validate access tokens. For now, you should understand that you have full flexibility in how you design the access token and lock down API access, to meet your business needs.

There is also a more subtle point about APIs only allowing requests that include valid access tokens. Figure 1-4 shows how each API downloads a cryptographic public key from the authorization server and uses it to verify access tokens. You should configure each API with a trusted URL to the authorization server that expresses a trust relationship. APIs do not trust each other. Instead, they only trust the authorization server.

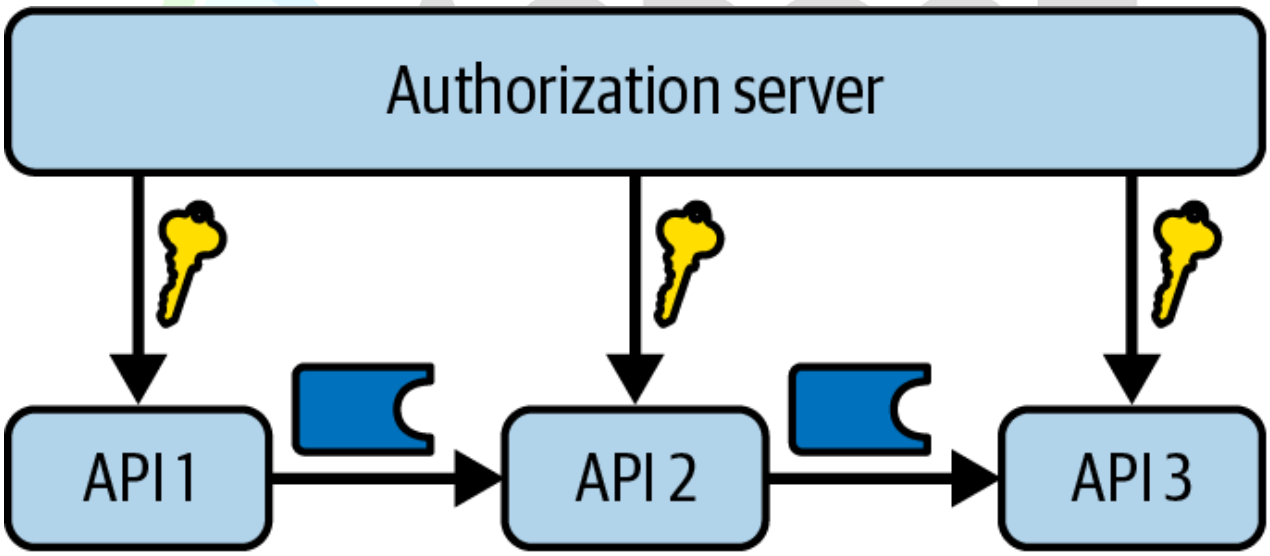

Figure 1-4. API trust